

Ammonia plant builder agent

Architecture and build plan

Prepared for Daemon · KNUST Chemical Engineering · scope: ammonia plant only

1. Goal

An agent that assembles an ammonia plant from prebuilt components, with a voice model that explains each part to the user as it's built. Scope is deliberately narrowed to ammonia only – a full production train from natural gas feed through to NH_3 product – which keeps the component list bounded and well-documented rather than open-ended.

2. Core design principle: separate physics from language

The two halves of this system have opposite requirements, and the architecture has to respect that:

- Assembling the plant and computing its numbers has to be **correct**. There is a right answer, governed by mass and energy conservation, and it must be checkable by a chemical engineer.
- Explaining parts by voice has to be **fluent**. There is no single right sentence, and the goal is clarity, not precision of derivation.

A free-form LLM agent that reasons its way to plant topology and numbers is the expensive, fragile choice for the first half – it can hallucinate a topology or produce numbers that don't actually satisfy conservation laws. The fix is not to remove the LLM, but to keep it out of the parts that have a right answer, and let it own the parts that don't.

3. System architecture

Layer	What it does	How it's built
Deterministic engine	Assembles the topology and computes the real mass/energy balance	Custom Python unit-op library plus a sequential-modular solver
Intake (LLM)	Natural-language spec to structured parameters	Single-shot, cheap/small model, JSON extraction only – not an agent loop
Base narration (LLM + TTS)	Turns real computed results into a spoken walkthrough	Script generated once, grounded in real numbers, then read by TTS – fixed, not regenerated live
What-if / interactive loop	User question changes a parameter; engine recomputes; result is narrated	Tool-calling agent: LLM proposes a parameter delta, engine computes it, result is real
Diagnosis / design rationale	Trade-off comparisons, fault scenarios, "why is X here" questions	LLM grounded in reference material/heuristics, not memory alone

4. Where the LLM actually adds value

"No LLM for the physics" does not mean no LLM. The LLM's job is everything language-shaped:

- 1 **Intake** – turning a natural-language spec into structured build parameters.
- 2 **Narration** – turning real engine output into a spoken explanation.
- 3 **The interactive loop** – interpreting a what-if question, calling back into the engine with the change, and narrating the real recomputed result. Every physical claim stays traceable to a tool call.
- 4 **Diagnosis and design rationale** – comparing trade-offs, explaining why a design looks the way it does, walking through fault scenarios, ideally grounded in retrieved reference material.
- 5 **Personalization** – same plant, different depth ("explain like I'm in year 1" vs. P&ID-level detail) is a framing job the LLM is well suited to.

5. Ammonia plant component library (MVP scope)

The full library needed for an ammonia plant is bounded – roughly eight or nine reusable unit-op models, each coded once and reused across any topology, not one engine per plant:

Component	Model type	Notes
Primary + secondary reformer	Reaction + heat transfer	Once-through
HT / LT shift reactors	Equilibrium reactor	Once-through
CO ₂ absorber	Simplified	Fixed removal efficiency for MVP
Methanator	Small equilibrium reactor	Once-through
Multi-stage compressor	Polytropic compression	Closed-form, no iteration needed
Synthesis converter	Equilibrium + fractional approach	60–70% approach to equilibrium for MVP; full Temkin–Pyzhev kinetics is a later upgrade
Heat exchangers / waste-heat boiler	Energy balance	
NH ₃ condenser / separator	Simplified vapour-liquid split	
Splitter (recycle / purge)	Pure mass balance	No physics – good first build to test plumbing

6. Process topology

Feed (natural gas + steam) enters a **syngas prep train** – reforming, shift, CO₂ removal, methanation – which is once-through with no recycle. Its output feeds the **synthesis loop**: compression, the converter, cooling, NH₃ separation, recycle of unconverted gas back to the converter inlet, and a purge bleed to prevent inert build-up. NH₃ product is drawn off the separator.

The syngas prep train is the easy half – every unit is once-through. The synthesis loop is the hard half, because the recycle stream only closes after iteration.

7. Build order

- 1 Define the stream object (temperature, pressure, flow, composition) – the currency every unit passes to the next.

- 2 Build the splitter/mixer – pure mass balance, validates the plumbing before any real physics.
- 3 Build the compressor – one closed-form equation, no iteration.
- 4 Build the shift reactors and methanator – equilibrium-based, single-pass, still no recycle.
- 5 Build the converter, separator, and the recycle loop – the one piece that needs iterative convergence (tear the recycle stream, solve with direct substitution or Wegstein).
- 6 Wire the full topology and validate against a published ammonia plant mass/energy balance, not just internal self-consistency.

Seven of nine unit ops are built and tested with plain sequential math before the hard convergence problem is ever touched.

8. The what-if sandbox

This is not a separate build – it reuses the same deterministic engine a second time, with one parameter changed:

- Build: `spec → parameters → engine.compute(params) → narrate(result)`
- What-if: "what if pressure is 250 bar" → `params.pressure = 250 → engine.compute(params) → narrate(diff between old and new result)`

The real cost is convergence speed, not the LLM: since a what-if question is usually a small perturbation of an already-converged state, **warm-starting** the solve from the previous converged values lets a small change converge in a couple of iterations instead of from scratch, keeping it responsive enough to feel conversational.

Narration for the what-if answer splits in two:

- **Structured parameter change** (e.g. a pressure bump) → template narration filled in from the engine's real before/after numbers. No LLM needed, cannot hallucinate.
- **Open-ended question** (e.g. "why does the purge stream exist") → genuinely needs an LLM, because the question itself is unbounded.

9. Voice narration approach

No custom voice model needs to be trained – that solves a harder problem than this needs. The base tour script is generated once, grounded in the real computed results, and reused on every playback. An off-the-shelf TTS engine reads it.

Option	Cost	Quality note
Self-hosted (e.g. Piper)	Near-zero marginal cost per generation	Lower voice quality
OpenAI TTS	Pay per character, moderate cost	Good quality
ElevenLabs / Azure	Pay per character, higher cost	Highest quality

10. Cost-saving decisions (summary)

- Deterministic code handles topology and physics – not an autonomous multi-step LLM agent loop.
- One cheap, single-shot LLM call per build for natural-language intake, instead of a multi-turn agent.

- Templated narration for structured what-if deltas, so most sandbox interactions cost no LLM call at all.
- Existing thermophysical property libraries (e.g. Cantera, CoolProp) instead of deriving thermodynamics from scratch.
- Self-hosted TTS as a lower-cost option if usage volume matters more than top-tier voice quality.

11. MVP simplifications

- Synthesis converter modelled as equilibrium-limited with a typical fractional approach to equilibrium (60–70%), rather than full multi-bed Temkin–Pyzhev kinetics.
- CO₂ absorber modelled with a fixed removal efficiency rather than detailed mass-transfer/absorption modelling.
- These are standard engineering simplifications, not shortcuts that compromise credibility – they can be upgraded to full kinetics/mass-transfer models later without changing the surrounding architecture.

12. Staged roadmap

Stage	What it adds
MVP	Spec in, plant assembled, 3D render, grounded voice walkthrough
v2	Live what-if sandbox with real recomputation, not canned answers
v3	Fault/training mode – inject a failure, agent walks through diagnosis like an operator trainer
v4	Exportable deliverables – a real mass-balance table or PFD the user can check or submit
v5	Comparative/economic mode – trade-offs between designs with real energy/capital/conversion numbers

The gap between MVP and v5 is not the voice or the 3D rendering – those already impress on day one. It is whether the what-if loop is real (recomputed) or fake (LLM describing a plausible-sounding consequence). That is the one thing worth protecting most carefully, since it is also the first thing a chemical engineer reviewing the app would test.

13. Open items to decide next

- Finalize the stream object schema (fields for temperature, pressure, flow, composition).
- Choose the specific correlations/equations for each unit-op model (e.g. Gillespie–Beattie for ammonia equilibrium, polytropic efficiency assumptions for the compressor).
- Confirm equilibrium-based vs. full-kinetics converter model for the first build.
- Pick a TTS provider based on the cost/quality trade-off above.